CEF440 · INTERNET PROGRAMMING · TASK 7

CheckAR

AI-Powered Smart Car Diagnostics — dashboard-light and engine-sound diagnosis without OBD-II hardware

GROUP 11

Funwi Kelsea Ndohnwi FE23A066

Ngadang Assonfack P. Estelle FE23A109

Ateh Swirri Fofang FE23A018

Tabot Ghislain Orock FE23A146

University of Buea · Department of Computer Engineering · Supervisor: Dr. Nkemeni Valery

CEF440 · INTERNET PROGRAMMING · TASK 7

FINAL PRESENTATION

University of Buea · Department of Computer Engineering · Supervisor: Dr. Nkemeni Valery

INTRODUCTION

Modern vehicles use dashboard warning lights and engine sounds to indicate faults.
However, many drivers:

- ✗ Do not understand warning symbols.
- ✗ Ignore unusual engine sounds.
- ✗ Delay vehicle maintenance.
- ✗ Depend entirely on mechanics.

This often results in:

- Expensive repairs
- Vehicle breakdowns
- Reduced vehicle lifespan
- Safety risks

THE PROBLEM

Diagnosing a fault shouldn't require a mechanic visit

Modern vehicles pack dozens of interdependent electronic systems. When a dashboard light flashes or an engine makes an unfamiliar sound, most car owners — especially across Cameroon and similar markets — have no way to interpret it themselves.

Unnecessary mechanic visits

Even minor, self-diagnosable issues get escalated, costing time and money.

Risk of exploitation

Non-technical owners are vulnerable to unscrupulous repairers.

Hardware-dependent tools

Existing OBD-II scanners need a physical dongle many older cars lack.

Two problems.
One unified fix.

Dashboard warning-light image
Photograph a lit icon; CheckAR recognises it instantly.

Engine-sound recording
Record up to 10 seconds; on-device ML classifies the fault.

No OBD-II dongle required
Works with just a smartphone camera and microphone.

OUR SOLUTION

WHAT IS CheckAR : “YOUR SMART CAR DOCTOR”

General objective: CheckAR is an AI-powered mobile application that:

- Scans dashboard warning lights.
- Records engine sounds.
- Detects vehicle faults.
- Provides repair suggestions.
- Shows maintenance tutorials.
- Works both online and offline.

WHAT IS CheckAR

CheckAR combines:

- Computer Vision
- Audio Classification
- Artificial Intelligence
- Mobile Technology

to provide vehicle diagnosis directly from a smartphone.

General & specific objectives

General objective: Design and specify CheckAR, an AI-powered app enabling car owners and mechanics to diagnose common faults from images and sound — no OBD-II hardware needed.

Task 1

Review mobile dev. approaches & pick an architecture

Task 2

Identify stakeholders & gather real requirements

Task 3

Analyze, prioritize & specify requirements (SRS)

Task 4

Model system scope & behaviour with diagrams

Task 5

Design a minimal, easy-to-implement UI/UX

Task 6

Design the data model & backend architecture

Why existing OBD-II scanner apps fall short

CAPABILITY	FIXD OBD2 SCANNER	CAR SCANNER ELM OBD2	CHECKAR
Works without a physical dongle	✗	✗	✓
Diagnoses from a photo of a warning light	✗	✗	✓
Diagnoses from engine sound	✗	✗	✓
Plain-language explanation + urgency level	Partial	Partial	✓
Mechanic discovery & tutorial links	✗	✗	✓

Insight: hardware-free, image- and sound-based diagnosis with plain-language guidance is the gap CheckAR is built to fill.

From interviews to a prioritized SRS

HOW REQUIREMENTS WERE GATHERED

- Structured interviews with owners & mechanics
- Bilingual (EN/FR) Google Forms surveys
- Group brainstorming sessions
- Reverse engineering of FIXD & Car Scanner

User-reluctance patterns uncovered

Accuracy-trust paradox · technical intimidation · safety/liability worries · poor lighting & connectivity

SAMPLE FUNCTIONAL REQUIREMENTS (MoSCoW)

ID	REQUIREMENT	PRIORITY
FR5	Capture dashboard images & detect warning-light icons	Must-Have
FR7	Record engine sound clips up to 10 seconds	Must-Have
FR8	Classify engine sounds into fault categories	Must-Have
FR13	Operate offline using on-device models	Must-Have
FR10	Link to relevant video tutorials per issue	Should-Have
FR17	Estimate urgency: Immediate / Soon / Monitor	Must-Have

Modelling scope and behaviour (Task 4)

Context Diagram

CheckAR exchanges data with the Admin, User, Google Maps, YouTube, and an Authentication Service.

Data Flow Diagram

Decomposes the system into authentication, image diagnosis, audio diagnosis, reporting, and maintenance-tracking processes.

Use Case Diagram

Two actors — Car User and Administrator — with include-relationships linking scanning, sound recording and reporting.

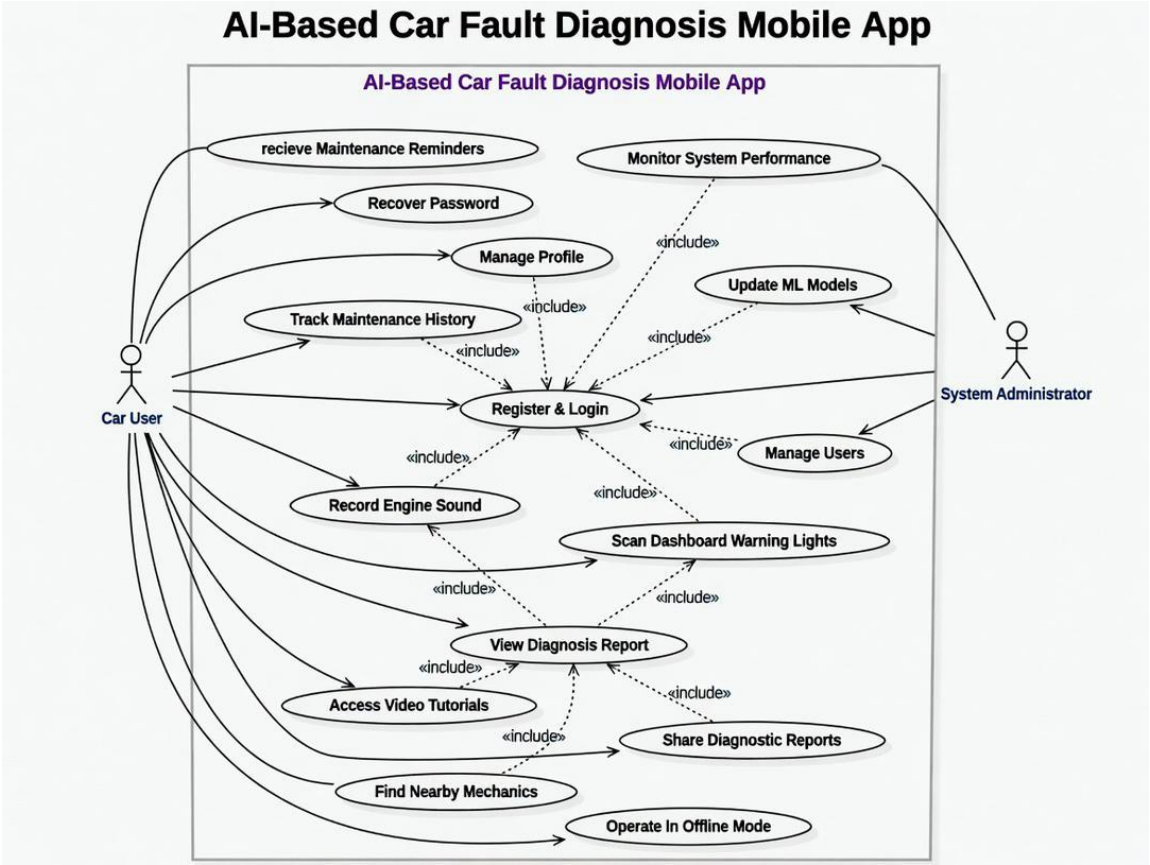


Figure 3.3: Use Case Diagram of CheckAR

A minimal, trust-building visual language

Brand identity centres on a professional blue (#2563EB) supported by a six-colour palette, the Inter font family, and a consistent grid — kept deliberately clean and easy to implement in Flutter. Sixteen screens were mocked up in total; the core flow is shown here.

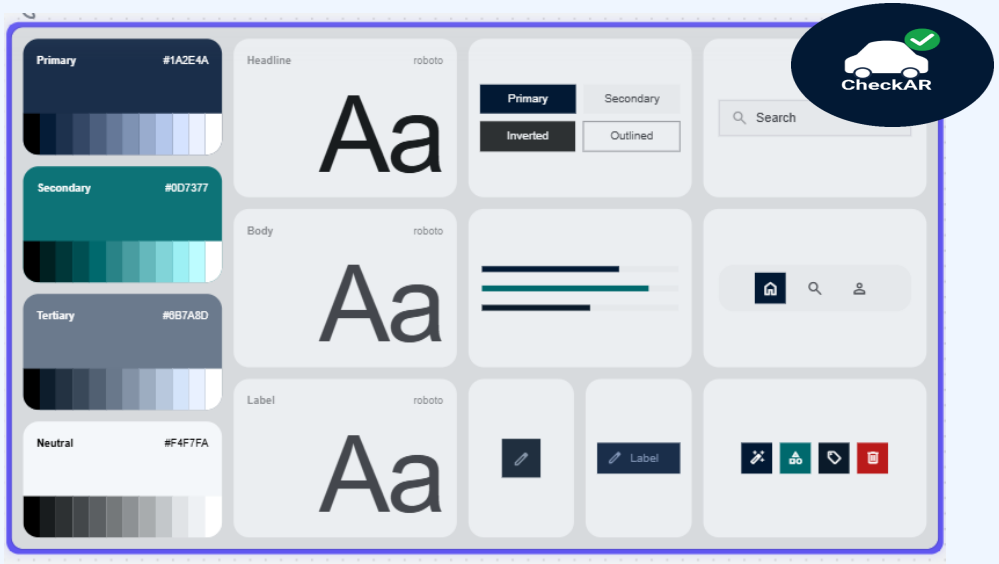
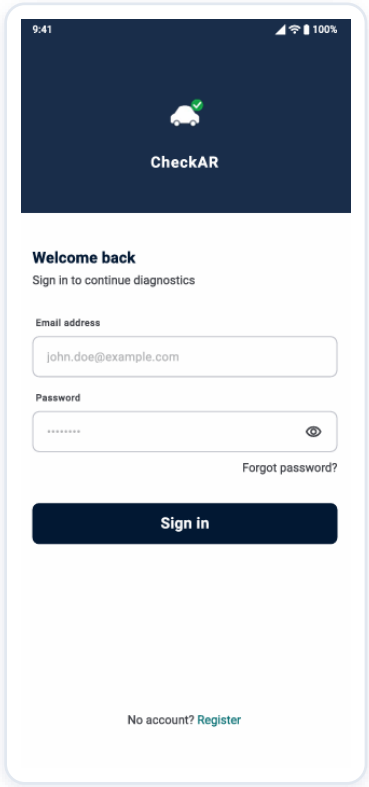
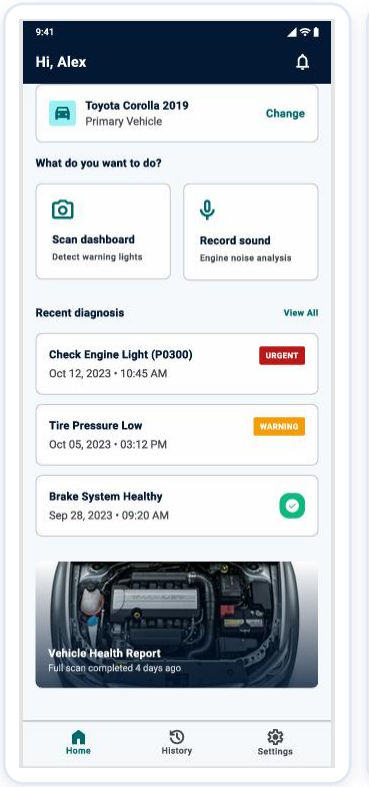


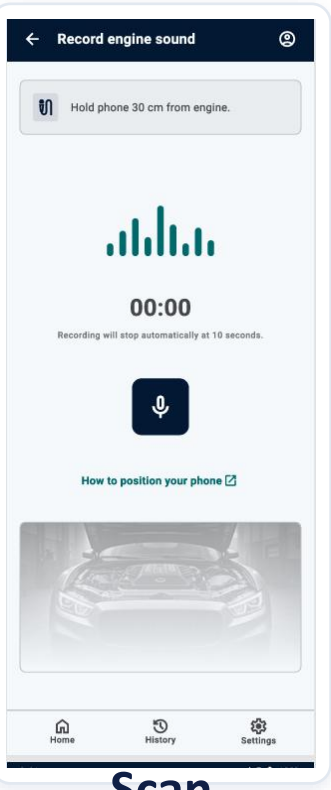
Figure 3.4 / 3.5 — CheckAR logo & colour palette



Login



Home



Scan Dashboard

Twelve-entity data model, three-layer backend

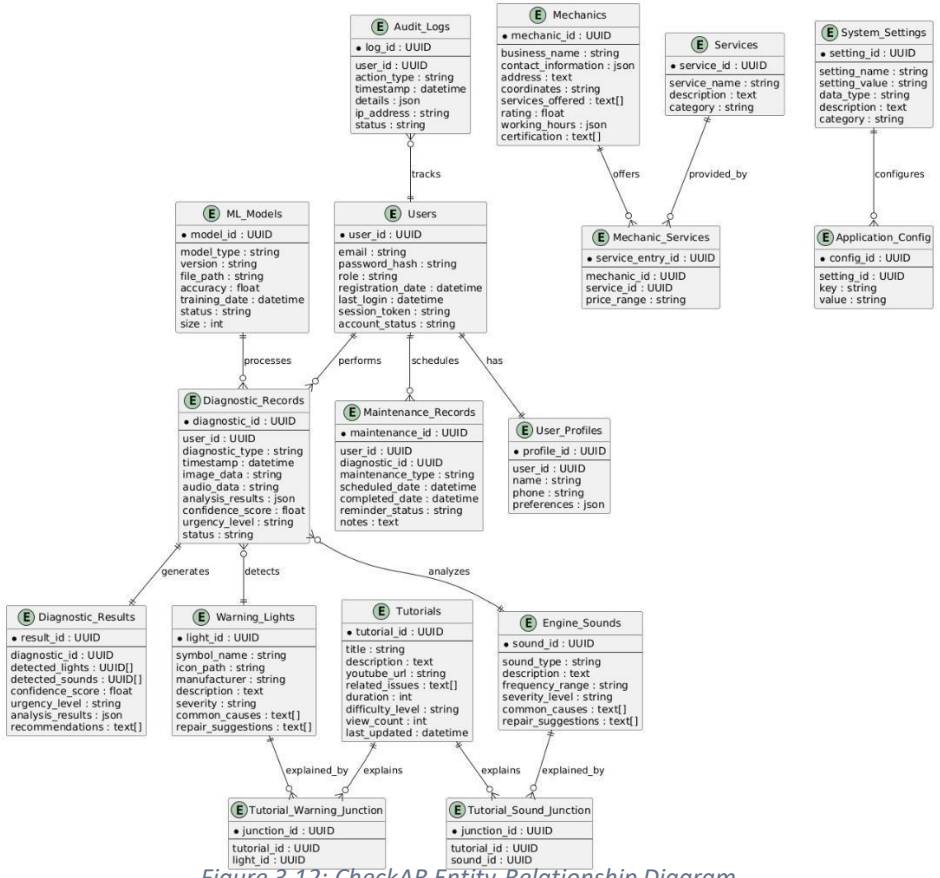


Figure 3.12: CheckAR Entity-Relationship Diagram

KEY ENTITIES

Users · User_Profiles · Diagnostic_Records · Diagnostic_Results · Warning_Lights · Engine_Sounds · Maintenance_Records · Mechanics · Services · Tutorials · ML_Models · Audit_Logs

Three-Layer Architecture

Flutter mobile client

↓ REST + JWT

Django / DRF backend API

↓ ORM

PostgreSQL relational database

Technology stack and progress to date

Flutter (Dart)

Cross-platform front-end

Django + DRF

Backend & REST API

PostgreSQL

Relational database

TensorFlow

Image & audio ML models

JWT Auth

Secure sessions

Docker

Containerized deployment

STATUS: DESIGN PHASE COMPLETE

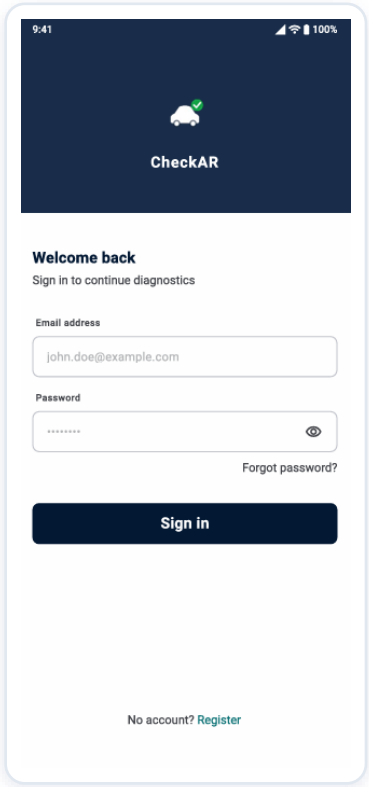
Objectives 1–6 (research, requirements, SRS, architecture, UI/UX, database design) are fully satisfied. The general objective — a working diagnostic app — awaits the implementation and testing phase, planned via prototype → alpha → beta → production testing with the original interview/survey participants.

WHY CHECKER IS DIFFERENT

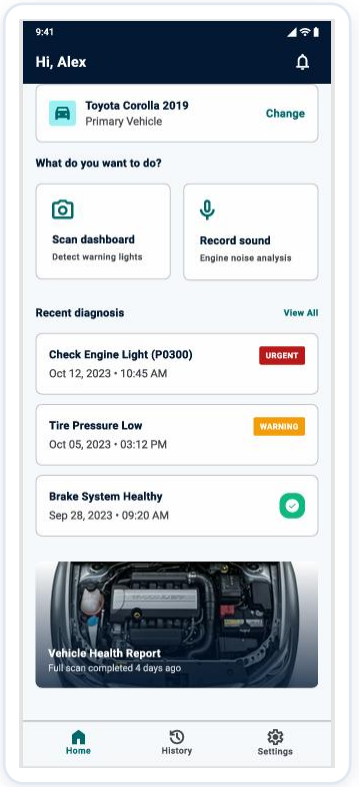
FEATURE	EXISTING APPS	CheckAR
Dashboard Recognition	✗	✓
Engine Sound Analysis	✗	✓
Video Tutorials	✗	✓
Offline Mode	x	✓
No External Hardware	✗	✓
AI Diagnosis	**Limited**	✓

SYSTEM OVERVIEW

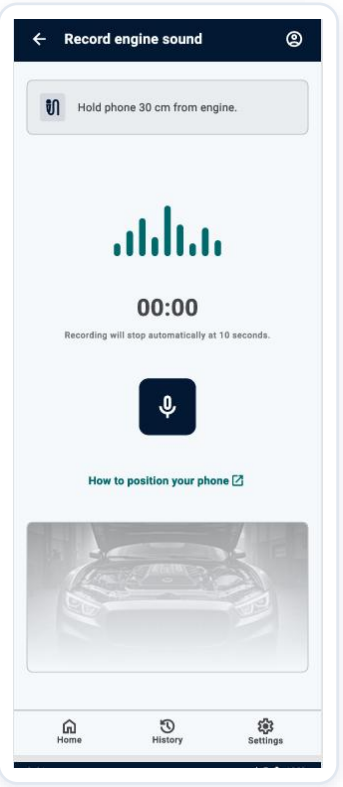
Step 1:
User scans dashboard light.
Step 2:
User records engine sound.
Step 3:
AI analyses the data.
Step 4:
Results are displayed.



Login



Home

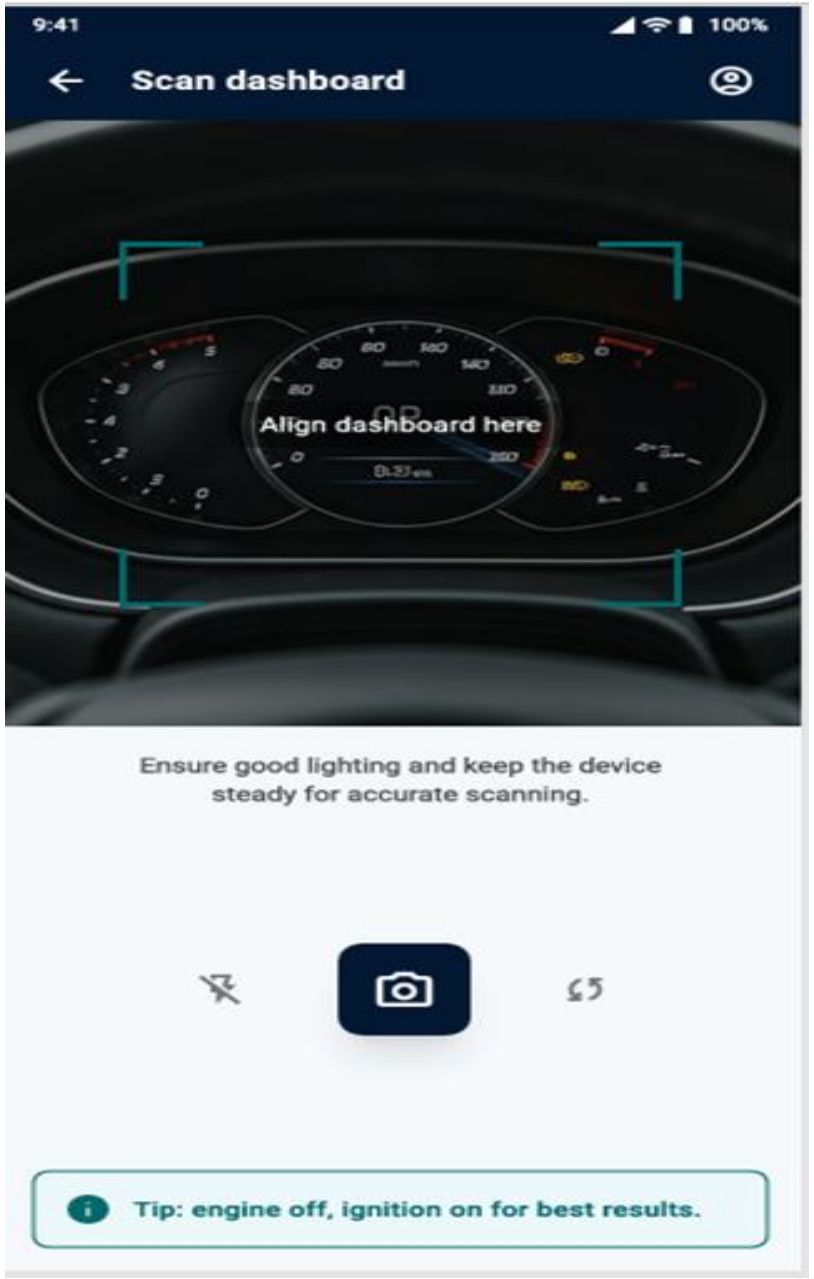


Scan Dashboard

DASHBOARD DIAGNOSIS

Application provides:

- ✓ Meaning
- ✓ Possible causes
- ✓ Urgency level
- ✓ Recommended actions



Scan Dashboard

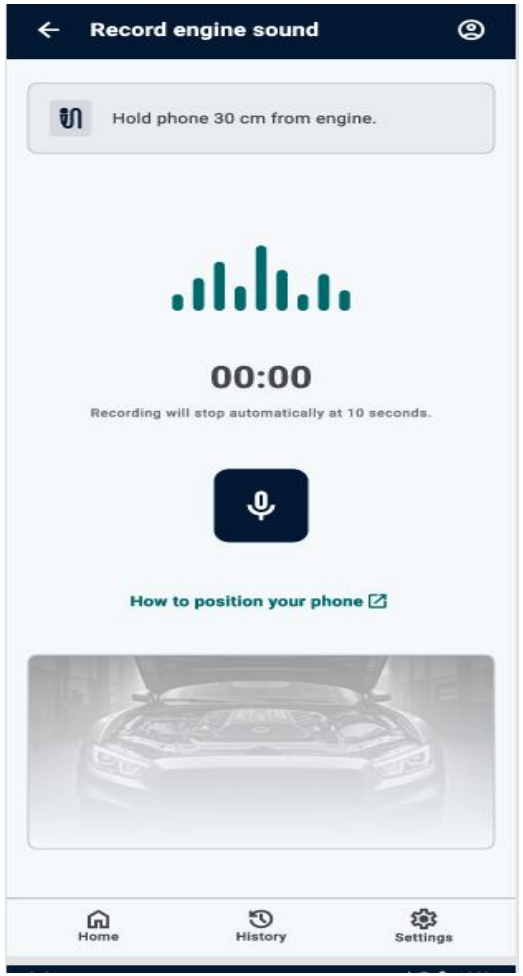
ENGINE SOUND ANALSIS

AI detects:

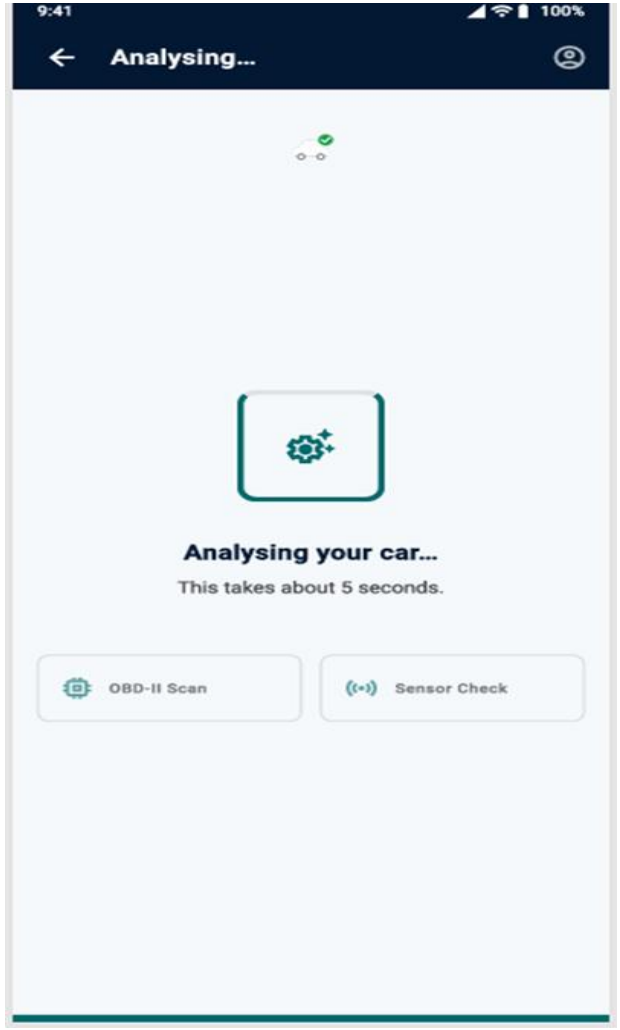
- ✓ Knocking
- ✓ Squealing
- ✓ Grinding
- ✓ Hissing
- ✓ Clicking

Then provides:

- ✓ Diagnosis
- ✓ Repair suggestions
- ✓ Maintenance tips



Record Dashboard

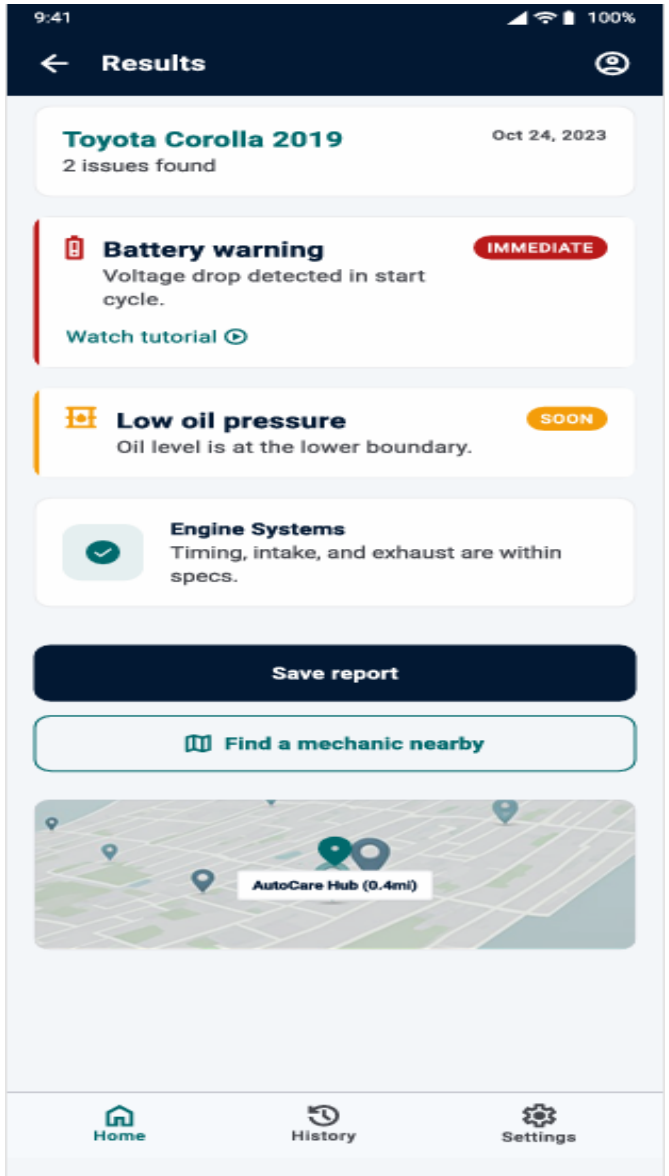


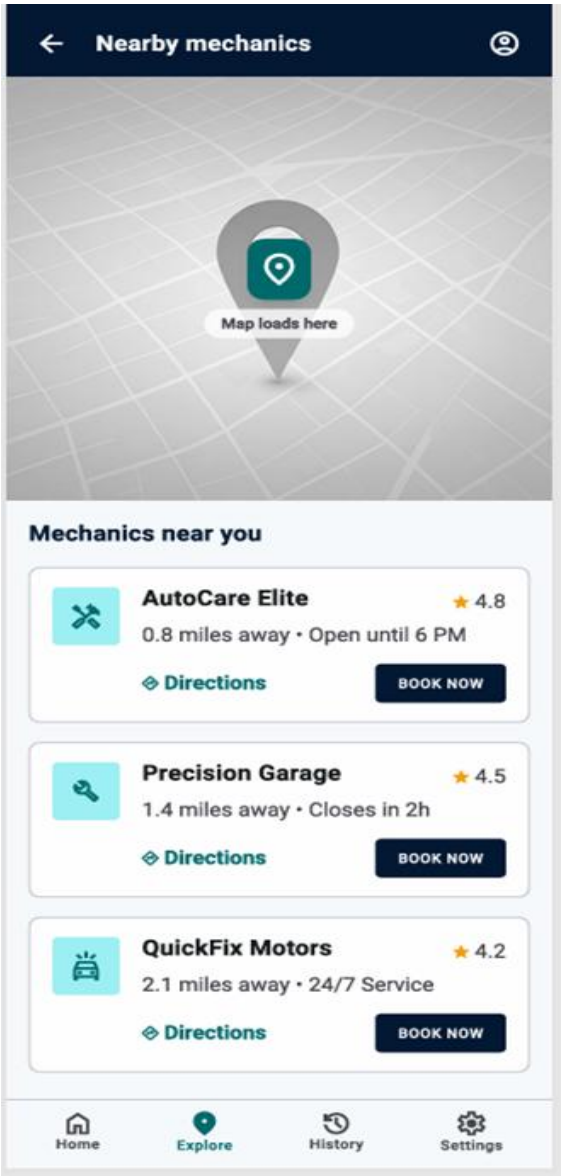
Analyzing Dashboard

RESULTS PAGE

Application provides:

- Fault explanation
- Confidence score
- Urgency level
- Nearby mechanics
- Video tutorials





Result Dashboard

TECHNOLOGIES USED

TECHNOLOGY	PURPOSE
Flutter	Mobile Development
Django	Backend
PostgreSQL	Database
TensorFlow	AI
GitHub	Version Control
Docker	Deployment
JWT	Security

WHAT WE LEARNED

IN INTERNET PROGRAMMING

We learned:

- ✓ REST APIs
- ✓ Backend Development
- ✓ Client-Server Architecture
- ✓ Authentication
- ✓ Database Connectivity
- ✓ System Security
- ✓ API Design

IN MOBILE PROGRAMMING

We learned:

- ✓ Mobile UI/UX Design
- ✓ Flutter Development
- ✓ Cross-platform Applications
- ✓ User Experience Design
- ✓ Mobile Architecture
- ✓ App Prototyping

TEAM COLLABORATION

Collaboration as a Team

Our team worked together by:

- ✓ Sharing responsibilities.
- ✓ Conducting meetings.
- ✓ Communicating frequently.
- ✓ Reviewing each other's work.
- ✓ Using GitHub for collaboration.

CHALLENGES / HOW WE OVERCAME THEM

Limitations Encountered

- Dataset collection difficulties.
- Limited AI training data.
- Integration challenges.
- Time constraints.
- Balancing features with simplicity.

This made our application:

- ✔ Better
- ✔ Stronger
- ✔ More realistic
- ✔ More user-centered

Through Proper Communication

We solved most challenges by:

- ☑ Team collaboration.
- ☑ Sharing ideas.
- ☑ Continuous meetings.
- ☑ Research and consultations.
- ☑ Proper planning.

CONCLUSION & FURTHER WORK

A complete blueprint, ready to build

CheckAR delivers an evidence-based, stakeholder-validated design for hardware-free vehicle diagnostics — grounded in real interviews and surveys with Cameroonian car owners and mechanics.

1. Implement the Flutter client and Django/DRF backend
2. Train & integrate TensorFlow models for image and sound classification
3. Run unit, integration, and user-acceptance testing
4. Deploy via Docker and beta-test the Android app with real users

---

Thank you — Group 11 | Funwi Kelsea · Ngadang Estelle · Ateh Swirri · Tabot Ghislain